МИНОБРНАУКИ РОССИИ

Санкт-Петербургский государственный

электротехнический университет

«ЛЭТИ» им. В.И. Ульянова (Ленина)

Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Построение и анализ алгоритмов»

Тема: Поиск с возвратом

Студент гр. 4343 **Лапшин А.Е.**

Преподаватель **Жангиров Т.Р.**

Санкт-Петербург

2026 г.

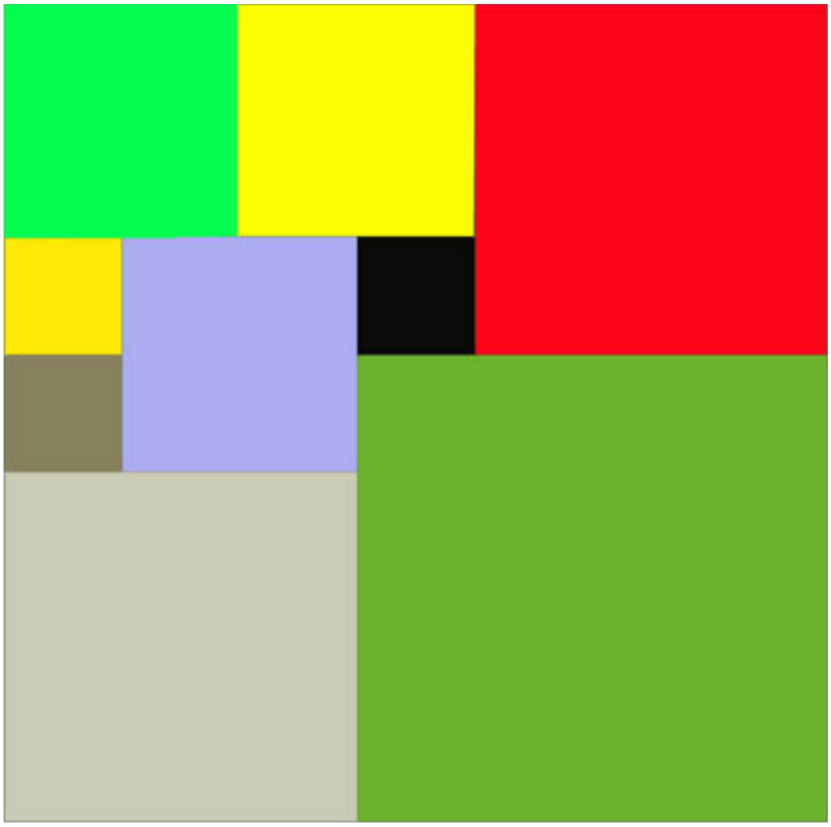
Вариант

Вар. 3и. Итеративный бэктрекинг. Исследование кол-ва операций от размера квадрата

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до N−1, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.



Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число N ($2 \le N \le 20$).

Выходные данные

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу (квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа x, y и w, задающие координаты левого верхнего угла ($1 \le x, y \le N$, $1 \le x, y \le N$) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Основные структуры данных

board: Двумерный вектор *vector<vector<int>>*, представляющий собой сетку размера *N x N*. Ячейка *board[i][j]* равна 1, если ячейка уже занята каким-либо квадратом, и 0, если свободна.

Square: Структура, хранящая координаты левого верхнего угла (*x, y*) и размер (*size*) установленного квадрата.

StateBoard: Структура, хранящая состояние доски (*board*) и список уже установленных квадратов (*placed_sq*) для конкретного узла в дереве поиска.

stack<StateBoard>: Стек для хранения состояний, которые нужно обработать.

Описание алгоритма

Для решения задачи был использован метод итеративного бэктрекинга (поиска с возвратом).

Пошаговое описание:

Инициализация:

- Считываем число N с консоли
- Если число четное, то очевидно, что науличшее решение, это заполнить квадрат 4-мя одинаковыми квадратами, для этого вызывается функция *FormedSolution(EvenSolution(N), N)*, которая формирует итоговый ответ для данного размера N.
- Создается начальное состояние start с пустой доской.
- Начальное состояние помещается в стек.
- Генерируется начальное "наилучшее" решение (InitBestSolution), чтобы алгоритм не тратил время на поиск решений, заведомо худших, чем это тривиальное. Для нечетных N это квадрат (N-1) в углу и полоска из единичных квадратов вдоль двух сторон.

<u>Основной цикл (пока стек не пуст):</u>

- Из стека извлекается текущее состояние current.
- Оптимизация 1 (по уже использованным квадратам): Поиск первой пустой клетки: Вызывается функция FullBoard, которая находит первую свободную клетку (x, y) слева направо, сверху вниз. Если свободных клеток нет, значит, решение найдено. Если количество квадратов в этом решении больше или равно текущего лучшего варианта размещения квадратов, то это состояние отбрасывается.
- Оптимизация 2 (оптимистичная оценка): Вычисляется оптимистичная оценка минимального количества оставшихся квадратов (количество пустых клеток делим на площадь минимально помещающегося квадрат). Если текущее количество квадратов + эта оценка больше или равно лучшего решения, состояние отсекается. Это помогает отбрасывать бесперспективные ветки решений на ранних этапах.
- Оптимизация 3 (определение минимальных и максимальных квадратов, которые мы будем использовать для заполнения квадрата)
  Определение максимального размера квадрата: Определяется максимально возможный размер квадрата, который можно поместить в свободное место, начиная с позиции (x, y). Он не может превышать N - x и N - y, а также быть больше N-2. Затем размер уменьшается до тех пор, пока не будет найден такой, который можно разместить в пустой области (CanPlace).
  Определение минимального размера квадрата: Если мы не поставили ни одного квадрата, то будем начинать с квадрата размером N/2, а иначе мы можем ставить квадраты с размерами стороны вплоть до 1.
- Генерация новых состояний: Для всех возможных размеров от максимального до минимального выполняется попытка размещения.

- Если квадрат можно разместить, создается новое состояние new_state1, в котором квадрат устанавливается, и это состояние помещается в стек.
- Оптимизация 4 (в лучшем решении есть некая закномерность, можно заметить, что, как минимум первые 3 квадрата, будут стоять в углу с определенным размером): Если текущее состояние было начальным (current.placed_sq.empty()), то для размера первого квадрата size также генерируется состояние new_state2, в котором сразу же размещаются три квадрата: первый (size), и два квадрата размером N-size в оставшихся углах. Это позволяет быстрее находить хорошие решения и сокращает перебор.

Сложность алгоритма

Временная сложность:

- Временная сложность составляет $\boldsymbol{O}((\boldsymbol{N}^3)^{\boldsymbol{D}})$, так как ячеек под квадраты N^2, а размеры от 1 до $N - 1$, где D - глубина стека. Так как максимальная глубина стека $D \leq N^2$ (в худшем случае все квадраты размера 1), то сложность получается $\boldsymbol{O}((\boldsymbol{N}^3)^{\boldsymbol{N}^2})$, но благодаря используемым оптимизациям, сложность на практике значительно снижается и не достигает данного значения.

Пространственная сложность:

- Основной расход памяти приходится на хранение стека состояний. Каждое состояние хранит копию доски (N^2 элементов). Таким образом, пространственная сложность составляет $\boldsymbol{O}(\boldsymbol{D}\boldsymbol{N}^2)$, где D - глубина стека. Так как максимальная глубина стека $D \leq N^2$ (в худшем случае все квадраты размера 1), то сложность получается $\boldsymbol{O}(\boldsymbol{N}^4)$.

Исследование

В рамках исследования был создан скрипт run_analysis.sh, который каждого значения N от 2 до 20 запускал алгоритм, и с помощью глобальной переменной operations_count подсчитывалось количество элементарных операций, результаты записывались в файл results.csv и затем для этих данных вызывался Python код (generate_graph.py), который строил график и сохранял его.

В программе элементарной операцией считается любое элементарное действие, выполняемое алгоритмом в процессе поиска решения (не учитывается формирование ответа и отладочная информация). К таким действиям относятся:

1. Проверка ячейки в циклах обхода доски (в функциях *CountEmptyCell*, *FullBoard*, *CanPlace*)
2. Установка квадрата (в *PlaceSquare* при заполнении каждой ячейки)
3. Проверка условий отсечения плохих веток событий
4. Создание нового состояния и помещение его в стек, а также извлечение состояния из стека

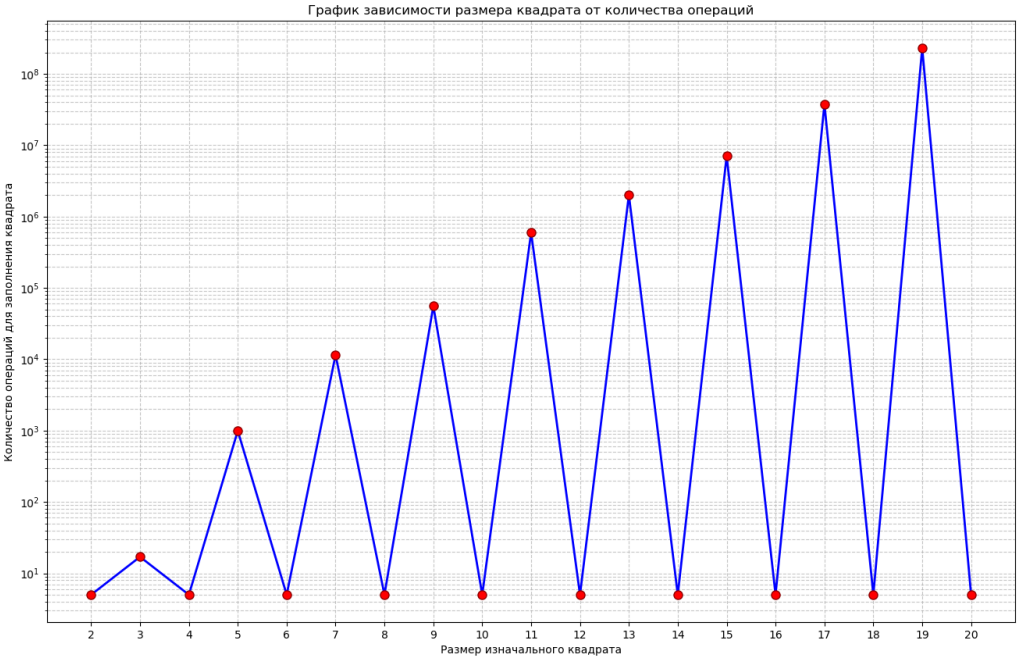


График зависимости размера квадрата от количества операций (логарифмическая шкала Y)

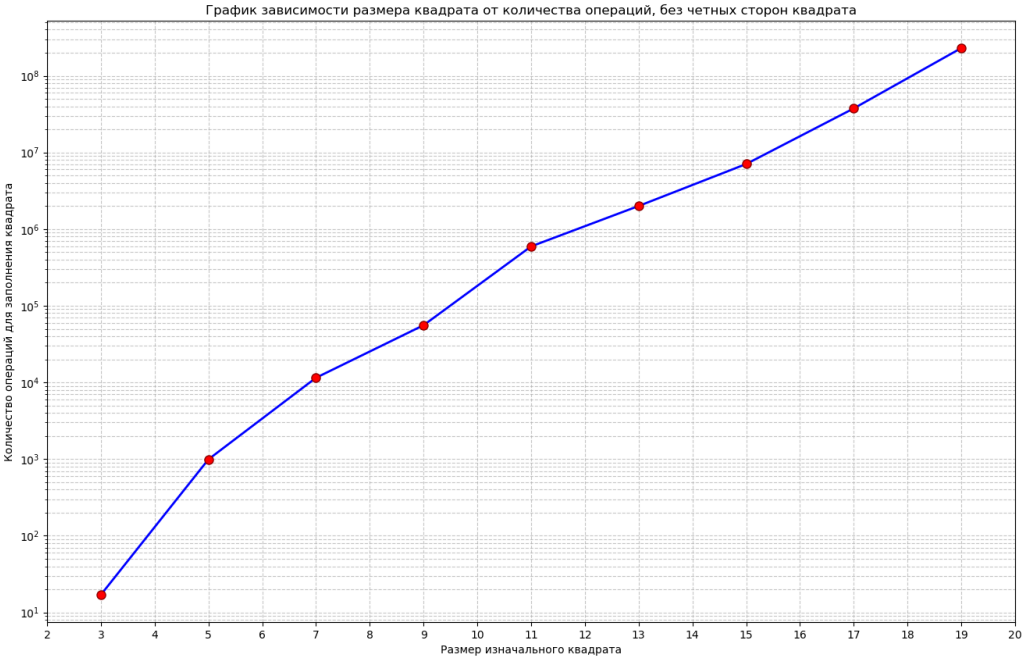


График зависимости размера квадрата от количества операций (логарифмическая шкала Y, без четных размеров сторон)

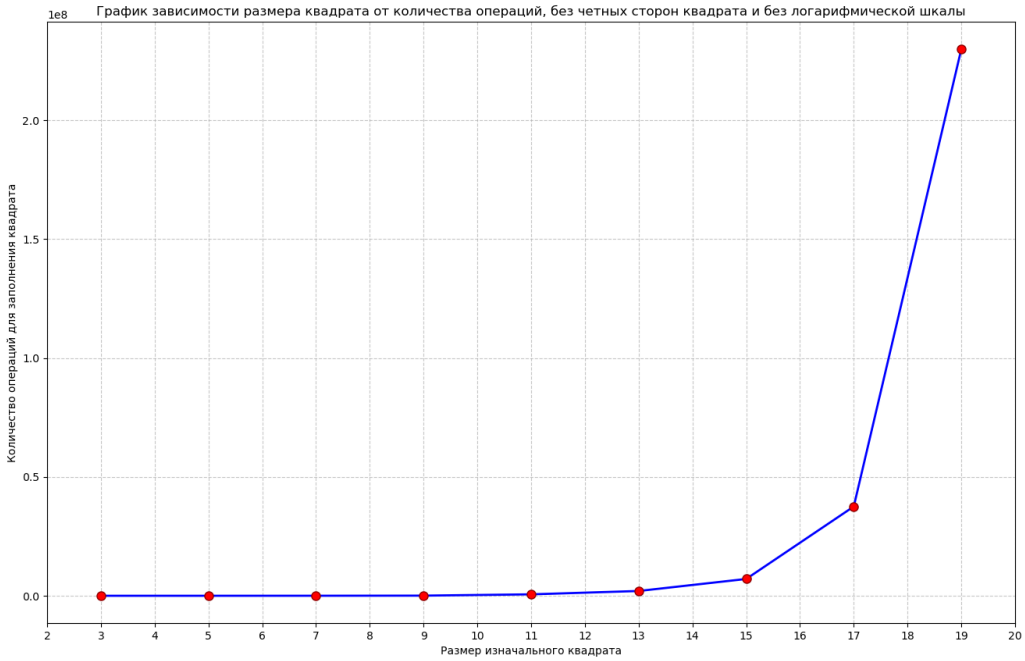


График зависимости размера квадрата от количества операций (не логарифмическая шкала Y, без четных размеров сторон)

Четные N: Для четных N задача решается тривиально (EvenSolution) за константное время O(1). На графике это видно по очень низким значениям, так как требуется 5 операций

1. **Нечетные N:** Для нечетных N задача является NP-трудной, и алгоритму приходится перебирать огромное количество комбинаций, чтобы доказать оптимальность решения. Это приводит к экспоненциальному росту числа операций.

Тестирование

1) Входные данные: 2
Выходные данные:

```
Запуск для N = 2
4
1 1 1
1 2 1
2 1 1
2 2 1


1 2
3 4
```

2) Входные данные: 3
Выходные данные:

```
Запуск для N = 3
6
1 1 2
3 1 1
3 2 1
3 3 1
1 3 1
2 3 1


1 1 5
1 1 6
2 3 4
```

3) Входные данные: 4
Выходные данные:

```
Запуск для N = 4
4
1 1 2
1 3 2
3 1 2
3 3 2


1 1 2 2
1 1 2 2
3 3 4 4
3 3 4 4
```

4) Входные данные: 5
Выходные данные:

```
Запуск для N = 5
8
1 1 3
4 1 2
1 4 2
```

```
3 4 2
4 3 1
5 3 1
5 4 1
5 5 1


1 1 1 3 3
1 1 1 3 3
1 1 1 4 4
2 2 5 4 4
2 2 6 7 8
```

5) Входные данные: 6
Выходные данные:

```
Запуск для N = 6
4
1 1 3
1 4 3
4 1 3
4 4 3


1 1 1 2 2 2
1 1 1 2 2 2
1 1 1 2 2 2
3 3 3 4 4 4
3 3 3 4 4 4
3 3 3 4 4 4
```

6) Входные данные: 7
Выходные данные:

```
Запуск для N = 7
9
1 1 4
5 1 3
1 5 3
4 5 1
4 6 2
5 4 1
5 5 1
6 4 2
6 6 2


1 1 1 1 3 3 3
1 1 1 1 3 3 3
1 1 1 1 3 3 3
1 1 1 1 4 5 5
2 2 2 6 7 5 5
2 2 2 8 8 9 9
2 2 2 8 8 9 9
```

7) Входные данные: 8

Выходные данные:

```
Запуск для N = 8
4
1 1 4
1 5 4
5 1 4
5 5 4


1 1 1 1 2 2 2 2
1 1 1 1 2 2 2 2
1 1 1 1 2 2 2 2
1 1 1 1 2 2 2 2
3 3 3 3 4 4 4 4
3 3 3 3 4 4 4 4
3 3 3 3 4 4 4 4
3 3 3 3 4 4 4 4
```

8) Входные данные: 9

Выходные данные:

```
Запуск для N = 9
6
1 1 6
7 1 3
1 7 3
4 7 3
7 4 3
7 7 3


1 1 1 1 1 1 3 3 3
1 1 1 1 1 1 3 3 3
1 1 1 1 1 1 3 3 3
1 1 1 1 1 1 4 4 4
1 1 1 1 1 1 4 4 4
1 1 1 1 1 1 4 4 4
2 2 2 5 5 5 6 6 6
2 2 2 5 5 5 6 6 6
2 2 2 5 5 5 6 6 6
```

9) Входные данные: 10

Выходные данные:

```
Запуск для N = 10
4
1 1 5
1 6 5
6 1 5
6 6 5


1 1 1 1 1 2 2 2 2 2
1 1 1 1 1 2 2 2 2 2
1 1 1 1 1 2 2 2 2 2
1 1 1 1 1 2 2 2 2 2
1 1 1 1 1 2 2 2 2 2
3 3 3 3 3 4 4 4 4 4
```

```
3 3 3 3 3 4 4 4 4 4
3 3 3 3 3 4 4 4 4 4
3 3 3 3 3 4 4 4 4 4
3 3 3 3 3 4 4 4 4 4
```

10) Входные данные: 11
Выходные данные:

```
Запуск для N = 11
11
1 1 6
7 1 5
1 7 5
6 7 1
6 8 1
6 9 3
7 6 1
7 7 2
8 6 1
9 6 3
9 9 3


1 1 1 1 1 1 3 3 3 3 3
1 1 1 1 1 1 3 3 3 3 3
1 1 1 1 1 1 3 3 3 3 3
1 1 1 1 1 1 3 3 3 3 3
1 1 1 1 1 1 3 3 3 3 3
1 1 1 1 1 1 4 5 6 6 6
2 2 2 2 2 7 8 8 6 6 6
2 2 2 2 2 9 8 8 6 6 6
2 2 2 2 2 10 10 10 11 11 11
2 2 2 2 2 10 10 10 11 11 11
2 2 2 2 2 10 10 10 11 11 11
```

11) Входные данные: 12
Выходные данные:

```
Запуск для N = 12
4
1 1 6
1 7 6
7 1 6
7 7 6


1 1 1 1 1 1 2 2 2 2 2 2
1 1 1 1 1 1 2 2 2 2 2 2
1 1 1 1 1 1 2 2 2 2 2 2
1 1 1 1 1 1 2 2 2 2 2 2
1 1 1 1 1 1 2 2 2 2 2 2
1 1 1 1 1 1 2 2 2 2 2 2
3 3 3 3 3 3 4 4 4 4 4 4
3 3 3 3 3 3 4 4 4 4 4 4
3 3 3 3 3 3 4 4 4 4 4 4
3 3 3 3 3 3 4 4 4 4 4 4
3 3 3 3 3 3 4 4 4 4 4 4
3 3 3 3 3 3 4 4 4 4 4 4
```

12) Входные данные: 13
Выходные данные:

```
Запуск для N = 13
11
1 1 7
8 1 6
1 8 6
7 8 2
7 10 4
8 7 1
9 7 3
11 10 1
11 11 3
12 7 2
12 9 2


1 1 1 1 1 1 1 3 3 3 3 3 3
1 1 1 1 1 1 1 3 3 3 3 3 3
1 1 1 1 1 1 1 3 3 3 3 3 3
1 1 1 1 1 1 1 3 3 3 3 3 3
1 1 1 1 1 1 1 3 3 3 3 3 3
1 1 1 1 1 1 1 3 3 3 3 3 3
1 1 1 1 1 1 1 4 4 5 5 5 5
2 2 2 2 2 2 6 4 4 5 5 5 5
2 2 2 2 2 2 7 7 7 5 5 5 5
2 2 2 2 2 2 7 7 7 5 5 5 5
2 2 2 2 2 2 7 7 7 8 9 9 9
2 2 2 2 2 2 10 10 11 11 9 9 9
2 2 2 2 2 2 10 10 11 11 9 9 9
```

13) Входные данные: 14
Выходные данные:

```
Запуск для N = 14
4
1 1 7
1 8 7
8 1 7
8 8 7


1 1 1 1 1 1 1 2 2 2 2 2 2 2
1 1 1 1 1 1 1 2 2 2 2 2 2 2
1 1 1 1 1 1 1 2 2 2 2 2 2 2
1 1 1 1 1 1 1 2 2 2 2 2 2 2
1 1 1 1 1 1 1 2 2 2 2 2 2 2
1 1 1 1 1 1 1 2 2 2 2 2 2 2
1 1 1 1 1 1 1 2 2 2 2 2 2 2
3 3 3 3 3 3 3 4 4 4 4 4 4 4
3 3 3 3 3 3 3 4 4 4 4 4 4 4
3 3 3 3 3 3 3 4 4 4 4 4 4 4
3 3 3 3 3 3 3 4 4 4 4 4 4 4
3 3 3 3 3 3 3 4 4 4 4 4 4 4
3 3 3 3 3 3 3 4 4 4 4 4 4 4
3 3 3 3 3 3 3 4 4 4 4 4 4 4
```

14) Входные данные: 15
Выходные данные:

```
Запуск для N = 15
6
1 1 10
11 1 5
1 11 5
6 11 5
11 6 5
11 11 5


1 1 1 1 1 1 1 1 1 1 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 4 4 4 4 4
1 1 1 1 1 1 1 1 1 1 4 4 4 4 4
1 1 1 1 1 1 1 1 1 1 4 4 4 4 4
1 1 1 1 1 1 1 1 1 1 4 4 4 4 4
1 1 1 1 1 1 1 1 1 1 4 4 4 4 4
2 2 2 2 2 5 5 5 5 5 6 6 6 6 6
2 2 2 2 2 5 5 5 5 5 6 6 6 6 6
2 2 2 2 2 5 5 5 5 5 6 6 6 6 6
2 2 2 2 2 5 5 5 5 5 6 6 6 6 6
2 2 2 2 2 5 5 5 5 5 6 6 6 6 6
```

15) Входные данные: 16
Выходные данные:

```
Запуск для N = 16
4
1 1 8
1 9 8
9 1 8
9 9 8


1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
```

16) Входные данные: 17

Выходные данные:

```
Запуск для N = 17
12
1 1 9
10 1 8
1 10 8
9 10 1
9 11 3
9 14 4
10 9 2
12 9 2
12 11 2
12 13 1
13 13 5
14 9 4


1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 4 5 5 5 6 6 6 6
2 2 2 2 2 2 2 2 7 7 5 5 5 6 6 6 6
2 2 2 2 2 2 2 2 7 7 5 5 5 6 6 6 6
2 2 2 2 2 2 2 2 8 8 9 9 10 6 6 6 6
2 2 2 2 2 2 2 2 8 8 9 9 11 11 11 11 11
2 2 2 2 2 2 2 2 12 12 12 12 11 11 11 11 11
2 2 2 2 2 2 2 2 12 12 12 12 11 11 11 11 11
2 2 2 2 2 2 2 2 12 12 12 12 11 11 11 11 11
2 2 2 2 2 2 2 2 12 12 12 12 11 11 11 11 11
```

17) Входные данные: 18

Выходные данные:

```
Запуск для N = 18
4
1 1 9
1 10 9
10 1 9
10 10 9


1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
```

```
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
```

18) Входные данные: 19

Выходные данные:

```
Запуск для N = 19
13
1 1 10
11 1 9
1 11 9
10 11 1
10 12 1
10 13 2
10 15 5
11 10 2
11 12 1
12 12 3
13 10 2
15 10 5
15 15 5
```

```
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3 3
1 1 1 1 1 1 1 1 1 1 4 5 6 6 7 7 7 7 7
2 2 2 2 2 2 2 2 2 8 8 9 6 6 7 7 7 7 7
2 2 2 2 2 2 2 2 2 8 8 10 10 10 7 7 7 7 7
2 2 2 2 2 2 2 2 2 11 11 10 10 10 7 7 7 7 7
2 2 2 2 2 2 2 2 2 11 11 10 10 10 7 7 7 7 7
2 2 2 2 2 2 2 2 2 12 12 12 12 12 13 13 13 13 13
2 2 2 2 2 2 2 2 2 12 12 12 12 12 13 13 13 13 13
2 2 2 2 2 2 2 2 2 12 12 12 12 12 13 13 13 13 13
2 2 2 2 2 2 2 2 2 12 12 12 12 12 13 13 13 13 13
2 2 2 2 2 2 2 2 2 12 12 12 12 12 13 13 13 13 13
```

19) Входные данные: 20

Выходные данные:

```
Запуск для N = 20
4
1 1 10
1 11 10
11 1 10
11 11 10
```

```
1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4
3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4
```

Программа для всех случаев отработала корректно.

Приложение

main.cpp:

```
#include <iostream>
#include <stack>
#include <algorithm>
#include <vector>
#include <fstream>

using namespace std;

long long operations_count = 0;

int CountEmptyCell(vector<vector<int>>& board) {
    int N = board.size();
    int count = 0;

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i][j] == 0) {
                count++;
            }
            operations_count++;
        }
    }
    return count;
}

bool FullBoard(vector<vector<int>>& board, int& x, int& y) {
    int N = board.size();

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            operations_count++;
            if (board[i][j] == 0) {
                x = i;
                y = j;
                return false;
            }
        }
    }
    return true;
}

bool CanPlace(vector<vector<int>>& board, int x, int y, int size) {
    int N = board.size();
    operations_count++;

    if (x + size > N || y + size > N) {
        return false;
    }
    for (int i = x ; i < x + size; i++) {
        for (int j = y ; j < y + size; j++) {
            operations_count++;
            if (board[i][j] == 1) {
                return false;
            }
        }
    }
    return true;
}
```

```
void PlaceSquare(vector<vector<int>>& board, int x, int y, int size) {
    for (int i = x ; i < x + size; i++) {
        for (int j = y ; j < y + size; j++) {
            operations_count++;
            board[i][j] = 1;
        }
    }
}

struct Square {
    int x, y, size;
};

struct StateBoard {
    vector<vector<int>> board;
    vector<Square> placed_sq;
};

void PrintBoard(int N, vector<Square>& solution) {
    cout << "\n\n";
    vector<vector<int>> board(N, vector<int>(N, 0));

    int number = 1;
    for (const auto& sq : solution) {
        int x = sq.x;
        int y = sq.y;
        int size = sq.size;
        for (int i = x; i < x + size; ++i) {
            for (int j = y; j < y + size; ++j) {
                board[i][j] = number;
            }
        }
        ++number;
    }

    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            cout << board[i][j] << " ";
        }
        cout << endl;
    }
}

void FormedSolution(vector<Square> solution, int N) {
    cout << solution.size() << endl;
    for (auto& square : solution) {
        cout << square.x + 1 << " "
            << square.y + 1 << " "
            << square.size << "\n";
    }
    PrintBoard(N, solution);
}

vector<Square> EvenSolution(int N) {
    vector<Square> solution;
    int half = N / 2;

    solution.push_back({0, 0, half});
    solution.push_back({0, half, half});
    solution.push_back({half, 0, half});
    solution.push_back({half, half, half});
    operations_count += 4;
```

```
    return solution;
}

vector<Square> InitBestSolution(int N) {
    vector<Square> solution;
    Square sq = {0, 0, N - 1};
    solution.push_back(sq);

    for (int i = 0; i < N; i++) {
        Square sq = {N - 1, i, 1};
        solution.push_back(sq);
        operations_count++;

    }

    for (int i = 0; i < N - 1; i++) {
        Square sq = {i, N - 1, 1};
        solution.push_back(sq);
        operations_count++;
    }
    return solution;
}


void write_results_to_file(int N, long long operations) {
    ofstream file("results.csv", ios::out);
    if (file.is_open()) {
        file << N << "," << operations << "\n";
        file.close();
    }
}


int main() {

    int N;
    cin >> N;

    if (N % 2 == 0) {
        FormedSolution(EvenSolution(N), N);
        operations_count++;
        write_results_to_file(N, operations_count);
        return 0;
    }

    stack<StateBoard> st;
    StateBoard start;
    start.board = vector<vector<int>>(N, vector<int>(N, 0));
    st.push(start);
    vector<Square> best_solution = InitBestSolution(N);


    while(!st.empty()) {
        StateBoard current = st.top();
        st.pop();

        int x, y;
        if (FullBoard(current.board, x, y)) {
            if (current.placed_sq.size() < best_solution.size()) {
                best_solution = current.placed_sq;
            }
            operations_count++;
```

```
                continue;
            }

            if (current.placed_sq.size() >= best_solution.size()) {
                operations_count++;
                continue;
            }
            int m = min(N - x, N - y);
            if (current.placed_sq.size() + CountEmptyCell(current.board) /
(m * m) >= best_solution.size()) {
                operations_count++;
                continue;
            }
            int min_size = (current.placed_sq.empty()) ? N / 2 : 0;
            int max_size = min({N - x, N - y, N - 2});
            while (max_size > 0 && !CanPlace(current.board, x, y, max_size))
{
                operations_count++;
                max_size--;
            }

            for(int size = max_size; size > min_size; size--) {

                operations_count++;

                if (CanPlace(current.board, x, y, size)) {
                    StateBoard new_state1 = current;
                    PlaceSquare(new_state1.board, x, y, size);
                    new_state1.placed_sq.push_back({x, y, size});
                    st.push(new_state1);
                    operations_count++;

                    #ifdef LOG
                    PrintBoard(N, new_state1.placed_sq);
                    cout << "Операций = " << operations_count << endl;
                    #endif
                    if (current.placed_sq.empty()) {
                        int second_size = N - size;

                        StateBoard new_state2 = current;
                        PlaceSquare(new_state2.board, 0, 0, size);
                        PlaceSquare(new_state2.board, size, 0, second_size);
                        PlaceSquare(new_state2.board, 0, size, second_size);
                        new_state2.placed_sq.push_back({x, y, size});
                        new_state2.placed_sq.push_back({size, 0,
second_size});
                        new_state2.placed_sq.push_back({0, size,
second_size});
                        st.push(new_state2);

                        operations_count += 3;

                        #ifdef LOG
                        PrintBoard(N, new_state2.placed_sq);
                        cout << "Операций = " << operations_count << endl;
                        #endif

                    }
                }
            }
        }

        write_results_to_file(N, operations_count);
```

```
    FormedSolution(best_solution, N);
    return 0;
}
```

generate_graph.py:

```
import matplotlib.pyplot as plt

x = []
y = []

with open('results.csv', 'r') as f:
    for line in f:
        parts = line.strip().split(',')
        if len(parts) >= 2:
            x.append(int(parts[0]))
            y.append(int(parts[1]))

print("ОСЬ X: ", " ".join(x))
print("ОСЬ Y: ", " ".join(y))


plt.figure(figsize=(16, 10))
plt.semilogy(x, y, 'bo-', linewidth=2, markersize=8,
markerfacecolor='red', markeredgecolor='darkred')
plt.xticks(x)
plt.xlabel('Размер изначального квадрата')
plt.ylabel('Количество операций для заполнения квадрата')
plt.title('График зависимости размера квадрата от количества операций')
plt.grid(True, linestyle='--', alpha=0.7, which='both')
plt.savefig("graph_result.png")
plt.show()
```

run_analysis.sh:

```
#!/bin/bash

g++ main.cpp -o program

for i in {1..20}
do
    echo "Запуск для N = $i"
    echo $i | ./program
done

python3 ./generate_graph.py

rm results.csv
```